PCC CS-503 · PYTHON PROGRAMMING

Installing Python

and running it seven ways

COURSE NOTES FOR LECTURE 3

Windows · macOS · Command Prompt · IDLE · Notepad
Jupyter Notebook · PyCharm · VS Code · VS Code + Jupyter

Dr. Aafaq

Dr. Aafaq
Department of Computer Science & Engineering
B.Tech. CSE, 5th Semester

Contents

3 Installing and Running Python **1**

3.1 The Big Picture . . . 1
3.2 Installing Python on Windows . . . 2
3.2.1 Using the python.org installer (recommended) . . . 2
3.2.2 Alternatives on Windows . . . 3
3.3 Installing Python on macOS . . . 3
3.3.1 Using the python.org installer (recommended) . . . 3
3.3.2 Alternative: Homebrew . . . 3
3.4 Anaconda: One Installer for Everything (Optional) . . . 4
3.5 pip and Virtual Environments in Two Minutes . . . 4
3.6 Our Test Program . . . 5
3.7 Way 1: Command Prompt / Terminal . . . 5
3.7.1 Interactive mode (the >>> prompt) . . . 5
3.7.2 Script mode: running a `.py` file . . . 6
3.8 Way 2: IDLE . . . 6
3.9 Way 3: Notepad (or TextEdit) + Command Prompt . . . 7
3.10 Way 4: Jupyter Notebook . . . 7
3.10.1 Install and start . . . 7
3.10.2 Working in a notebook . . . 8
3.11 Way 5: PyCharm . . . 8
3.11.1 Install . . . 9
3.11.2 First project and run . . . 9
3.12 Way 6: Visual Studio Code . . . 9
3.12.1 Install . . . 9
3.12.2 First run . . . 10
3.13 Way 7: VS Code with Jupyter Notebooks . . . 10
3.14 Choosing a Tool . . . 11
3.15 Troubleshooting . . . 12
3.16 Summary . . . 12

Dr. Aafaq

CHAPTER 3

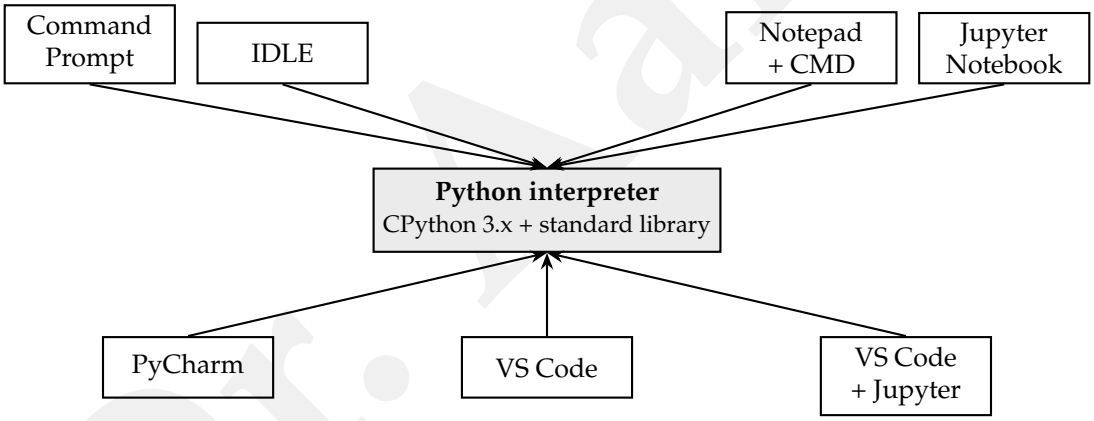
Installing and Running Python

How to use these notes

This lecture is a hands-on lab guide. Do every numbered step on your own laptop or the lab PC as you read. Lines that start with `C:\>` are typed in the Windows *Command Prompt*; lines that start with `%` are typed in the macOS *Terminal*; lines that start with `>>>` are typed at the Python prompt. Do not type the prompt itself. Keys are shown as `Ctrl+S`; on a Mac read `Ctrl` as `Cmd` unless the text says otherwise.

3.1 The Big Picture

Every tool in this lecture, from the black Command Prompt window to VS Code, does the same job underneath: it hands your code to the **Python interpreter** (`python.exe` on Windows, `python3` on macOS) and shows you what comes back. Install the interpreter *once*; then pick whichever front end suits the task.



Key idea

There is one Python and many windows onto it. If `python --version` works in the Command Prompt or Terminal, every other tool can be made to work too; if it fails, fix that first.

Which Python should I install?

- **Python 3, the current stable release** (3.14 or newer in 2026). Any 3.12+ is fine for this course. Never install Python 2; it reached end of life in 2020.
- **64-bit.** On Windows choose the 64-bit (x86-64) installer; on a Windows ARM laptop choose ARM64. On macOS the python.org installer is *universal2* and runs natively on both Intel and Apple Silicon (M1–M4) Macs.
- **python.org or Anaconda?** python.org gives a small, clean Python; you add libraries with `pip` as you need them. Anaconda is a large bundle (Python + Jupyter + NumPy + Pandas + Matplotlib + hundreds more) and is convenient for data work. This course uses python.org Python as the default and shows Anaconda in Section 3.4. Installing both is allowed but confusing for beginners; pick one.

Is Python already installed?

Open a terminal (Section 3.7 explains how) and type:

```
C:\> python --version          # Windows
C:\> py --version              # Windows launcher
% python3 --version            # macOS
Python 3.14.0
```

If you see a version number of 3.12 or higher you may skip the installation sections. On Windows, if typing `python` *opens the Microsoft Store* instead, Python is *not* installed: that is only a shortcut (see Troubleshooting).

3.2 Installing Python on Windows

3.2.1 Using the python.org installer (recommended)

Step 1. Open `https://www.python.org/downloads/` in a browser. The yellow button offers the latest Python for Windows. Click it to download the installer (`python-3.14.x-amd64.exe`, about 30 MB).

Step 2. Double-click the downloaded file. On the **first screen**, before anything else:

- tick **Add python.exe to PATH** *(the most important tick in the whole course)*;
- tick **Use admin privileges when installing py.exe** if it is shown.

Step 3. Click **Install Now**. Accept the Windows security prompt (*Do you want to allow this app to make changes?* → Yes).

Step 4. On the final screen, if you see **Disable path length limit**, click it (it lets Python use long folder names), then **Close**.

Step 5. Verify. Open a *new* Command Prompt (old windows do not see the new PATH) and type:

```
C:\> python --version
Python 3.14.0
C:\> pip --version
pip 25.2 from C:\Users\you\AppData\Local\...\Python314\...
C:\> where python
C:\Users\you\AppData\Local\Programs\Python\Python314\python.exe
```

Installed item	What it is
`python.exe`	The interpreter itself
`py.exe`	The *Python launcher*: `py` runs the newest Python, `py -3.12` a specific one, `py -0` lists all installed versions
`pip`	Package installer for third-party libraries (Section 3.5)
IDLE	Python's own small editor and shell (Start menu → IDLE (Python 3.14 64-bit))
Python docs	Offline manual (Start menu → Python 3.14 Manuals)

Watch out: forgot to tick "Add to PATH"?

Then `python` is "not recognized" in the Command Prompt. Easiest fix: run the installer again, choose **Modify** → Next → tick **Add Python to environment variables** → Install. Meanwhile `py` usually still works, because the launcher lives in the Windows folder.

Dr. Aarad

3.2.2 Alternatives on Windows

- **Python Install Manager.** From Python 3.14, python.org also offers a small manager (also on the Microsoft Store as *Python Install Manager*). After installing it, `py install 3.14` downloads a version and `py list` shows what you have. It is the direction python.org is moving in; either route gives the same interpreter.
- **Microsoft Store "Python 3.x" app.** Works for simple scripts but runs in a sandbox that can confuse some tools. Prefer python.org for this course.
- **winget** (for those comfortable with the command line): `winget install Python.Python.3.14`.

3.3 Installing Python on macOS

macOS no longer ships a usable Python for you (Python 2 was removed in macOS 12.3). Typing `python3` on a fresh Mac may pop up a dialog offering to install the *Command Line Developer Tools*; that gives an older Python meant for Apple's tools. Install your own instead.

3.3.1 Using the python.org installer (recommended)

Step 1. Open `https://www.python.org/downloads/` in Safari. Click **Download Python 3.14.x**; you get a `.pkg` file (macOS 64-bit universal2 installer).

Step 2. Double-click the `.pkg` and click **Continue** through Introduction, Read Me and License (**Agree**), then **Install**. Enter your Mac password when asked.

Step 3. When it finishes, a Finder window opens at Applications > Python 3.14. Double-click **Install Certificates.command**. A Terminal window runs and closes. (Skipping this causes `SSL: CERTIFICATE_VERIFY_FAILED` errors when your programs or `pip` download from the internet.)

Step 4. **Verify.** Open **Terminal** (`Cmd+Space`, type *Terminal*, `Return`) and type:

```
% python3 --version
Python 3.14.0
% which python3
/Library/Frameworks/Python.framework/Versions/3.14/bin/python3
% pip3 --version
pip 25.2 from /Library/Frameworks/Python.framework/...
```

Key idea: `python3`, not `python`, on a Mac

On macOS the commands are `python3` and `pip3`. Plain `python` usually says "command not found". Everything in these notes written as `python` should be read as `python3` on a Mac (or inside an activated virtual environment, where both work).

3.3.2 Alternative: Homebrew

Developers who already use the Homebrew package manager can type `brew install python`. It installs `python3` under `/opt/homebrew` (Apple Silicon) or `/usr/local` (Intel). Homebrew's Python does not include IDLE's Tk support by default (`brew install python-tk` adds it). For beginners the python.org installer is simpler.

Dr. Afia

3.4 Anaconda: One Installer for Everything (Optional)

Anaconda bundles Python, Jupyter, Spyder and the scientific libraries we use in Unit IV (NumPy, Pandas, Matplotlib) in one installer.

1. Download the **Anaconda Distribution** installer for your OS from `https://www.anaconda.com/download` (about 1 GB; a free account sign-in is optional, *Skip registration* works). **Miniconda** is the small alternative: Python + `conda` only.
2. Run it with the defaults: *Just Me*, default folder. On Windows leave "Add Anaconda to my PATH" *unticked*, as the installer advises.
3. Start it from **Anaconda Navigator** (a launcher with buttons for Jupyter, VS Code, Spyder) or, on Windows, **Anaconda Prompt** from the Start menu. On a Mac the normal Terminal works, and its prompt starts with `(base)`.

```
(base) C:\> python --version
Python 3.13.5
(base) C:\> conda install numpy     # Anaconda's package manager
```

Note

Anaconda's Python is usually one version behind python.org. That is fine. Inside Anaconda, prefer `conda install`; use `pip install` only for packages conda does not have. Organisations with more than 200 employees need a paid licence for Anaconda's default channel; students and universities are covered by the free terms, and Miniconda with the `conda-forge` channel is free for everyone.

3.5 pip and Virtual Environments in Two Minutes

pip installs third-party packages from PyPI (`https://pypi.org`). The safest way to call it is through the Python you mean, with `-m`:

```
C:\> python -m pip install numpy matplotlib   # Mac: python3 -m pip
C:\> python -m pip list                          # what is installed
C:\> python -m pip install --upgrade pip         # update pip itself
```

A **virtual environment** (venv) is a private copy of Python for one project, so that its packages do not clash with another project's. PyCharm and VS Code create one for you; by hand it is:

```
C:\proj> python -m venv .venv            % python3 -m venv .venv
C:\proj> .venv\Scripts\activate          % source .venv/bin/activate
(.venv) C:\proj> pip install requests     # goes into .venv only
(.venv) C:\proj> deactivate
```

(Windows on the left, macOS on the right.) If PowerShell refuses to run `activate` with a "running scripts is disabled" error, use the Command Prompt instead, or run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.

Key idea

`ModuleNotFoundError` after a successful `pip install` almost always means the package went into a *different* Python than the one running your code. Use `python -m pip` and check the interpreter your editor has selected.

3.6 Our Test Program

We will run the same small file in every tool, so you can compare them fairly. Type it exactly; the four-space indents matter.

Listing 3.1: `hello.py`

```
# hello.py -- Lecture 3 test program
import sys, platform

name = input('Your name: ')
print(f'Hello, {name}! Welcome to Python.')
print('Python version :', platform.python_version())
print('Running from   :', sys.executable)

total = 0
for i in range(1, 6):
    total += i * i
print('1^2 + ... + 5^2 =', total)
```

```
Your name: Asha
Hello, Asha! Welcome to Python.
Python version : 3.14.0
Running from   : C:\Users\you\AppData\...\Python314\python.exe
1^2 + ... + 5^2 = 55
```

The `sys.executable` line is a diagnostic trick: it tells you *which* Python on the machine ran the file. Keep it in mind for when packages "disappear".

Where to save your work

Make one folder for the course, e.g. `C:\Users\you\python-lab` on Windows or `~/python-lab` on macOS, and keep every file there. Avoid spaces and OneDrive/iCloud-synced folders for code if you can.

3.7 Way 1: Command Prompt / Terminal

The terminal is the most basic way, and the one every other tool falls back on.

Opening it. **Windows:** press `Win+R`, type `cmd`, `Enter` (or Start → type *Command Prompt*). PowerShell and Windows Terminal work the same way for our purposes. **Tip:** in File Explorer, click in the address bar of your folder, type `cmd` and press `Enter` to open a Command Prompt already *in* that folder. **macOS:** `Cmd+Space`, type *Terminal*, `Return`. In Finder, right-click a folder → Services > New Terminal at Folder.

3.7.1 Interactive mode (the >>> prompt)

```
C:\> python
Python 3.14.0 (tags/v3.14.0, ...) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3 * 4
14
>>> name = 'CSE'
>>> print(name * 3)
CSECSECSE
>>> exit()
C:\>
```

Results of expressions are echoed automatically. Leave with `exit()`, or Ctrl+Z then Enter on Windows, Ctrl+D on macOS. Since Python 3.13 the prompt is in colour, supports multi-line editing with the ↑ key, and accepts plain `exit` without brackets.

3.7.2 Script mode: running a `.py` file

```
C:\> cd C:\Users\you\python-lab       % cd ~/python-lab
C:\Users\you\python-lab> dir          % ls
hello.py
C:\Users\you\python-lab> python hello.py    % python3 hello.py
Your name: ...
```

Windows CMD	macOS Terminal	Meaning
`cd folder`	`cd folder`	change directory (`cd ..` goes up)
`dir`	`ls`	list files
`cls`	`clear`	clear the screen
`python f.py`	`python3 f.py`	run a script
`python -i f.py`	`python3 -i f.py`	run, then stay at `>>>` to inspect variables
`python -c "print(7*6)"`	`python3 -c "print(7*6)"`	run one line
`python f.py > out.txt`	`python3 f.py > out.txt`	save the output in a file
Ctrl+C	Ctrl+C	stop a running program

Double-clicking a `.py` file in Explorer also runs it, but the window closes the instant the program ends (or crashes), so you see nothing. End scripts with `input('Press Enter to exit')` if you want to run them this way. On macOS double-click opens the file in IDLE instead.

> **Try it**
> Run `python -i hello.py`. After it finishes, type `total` and `name` at the `>>>` prompt. The variables are still alive: `-i` is a cheap debugger.

3.8 Way 2: IDLE

IDLE (Integrated Development and Learning Environment) comes free with python.org Python. It has two kinds of windows: a **Shell** window (the `>>>` prompt) and **Editor** windows for files.

Opening it. **Windows:** Start → type *IDLE* → IDLE (Python 3.14 64-bit). **macOS:** Applications > Python 3.14 > IDLE (or Spotlight: *IDLE*). From a terminal: `python -m idlelib` (`python3 -m idlelib`). Anaconda installs also contain IDLE (`python -m idlelib`).

Step 1. The Shell window opens with a `>>>` prompt. Try `2**10`. Use Alt+P / Alt+N (Mac: Ctrl+P / Ctrl+N) to recall previous / next commands.

Step 2. File > New File (Ctrl+N) opens an editor. Type `hello.py`. IDLE colours keywords and auto-indents after a colon.

Step 3. File > Save (Ctrl+S); choose your `python-lab` folder and the name `hello.py`. IDLE adds `.py` if you omit it.

Draft

Step 4. Run > Run Module or F5. The Shell shows `= RESTART: ...hello.py =` and the program runs there; type your name when asked.

Step 5. After the run, the program's variables remain in the Shell: type `total`.

Useful extras: Format > Indent/Dedent Region (Ctrl+] / Ctrl+[), Format > Comment Out Region (Alt+3), Tab for name completion, and Debug > Debugger for single-stepping. Options > Configure IDLE changes the font size for projector use.

Watch out

F5 restarts the Shell each time, clearing variables you typed by hand. Also, do not name your own file after a standard module (`random.py`, `turtle.py`, `math.py`): your file will be imported instead of the real module, and IDLE itself may fail to start.

3.9 Way 3: Notepad (or TextEdit) + Command Prompt

Any plain-text editor can write Python; you then run the file from the terminal. This shows clearly that a program is *just a text file*.

Windows: Notepad

Step 1. Start → *Notepad*. Type `hello.py`. Use four spaces for each indent (Notepad's Tab inserts a tab character; mixing tabs and spaces gives `TabError`).

Step 2. File > Save As. Go to `python-lab`. Set **Save as type: All files (*.*)**, **Encoding: UTF-8**, and type the name `hello.py`. (Typing the name *with quotes*, `"hello.py"`, also forces the extension.)

Step 3. In File Explorer turn on View > Show > File name extensions once, so you can check the file is `hello.py` and not `hello.py.txt`.

Step 4. Open a Command Prompt in that folder and run `python hello.py`.

Step 5. Edit in Notepad, Ctrl+S, press ↑ Enter in the Command Prompt to run again. This edit–save–run loop is how programmers worked for decades.

macOS: TextEdit

TextEdit saves rich text by default, which Python cannot read. Before typing, choose Format > Make Plain Text (Shift+Cmd+T); to make this permanent, TextEdit > Settings > New Document > Plain text. Also turn off Edit > Substitutions > Smart Quotes, otherwise `'` becomes a curly quote and Python reports `SyntaxError: invalid character`. Save as `hello.py` (if asked, choose *Use .py*) and run it with `python3 hello.py` in Terminal.

Note

Notepad is fine for a first program but has no syntax colours, no auto-indent and no error checking. Better free plain editors: Notepad++ (Windows), Sublime Text, or just IDLE. Never write code in Word: it saves `.docx`, not text.

3.10 Way 4: Jupyter Notebook

A **notebook** (`.ipynb` file) mixes runnable code **cells**, their outputs (text, tables, plots) and formatted **Markdown** notes in one document that opens in your web browser. It is the standard tool for data analysis, lab records and teaching.

3.10.1 Install and start

Dr. Afaq

```
C:\> python -m pip install notebook     # once (not needed in Anaconda)
C:\> cd C:\Users\you\python-lab
C:\Users\you\python-lab> jupyter notebook   # macOS: same, in Terminal
    To access the server, open this file in a browser: ...
    http://localhost:8888/tree?token=3f9c...
```

A browser tab opens at `localhost:8888` showing the folder you started from. Anaconda users can instead click **Jupyter Notebook** in Anaconda Navigator. If `jupyter` is "not recognized", use `python -m notebook`.

Watch out

The black terminal window *is* the Jupyter server. Keep it open while you work; closing it kills your notebooks. Stop the server with `Ctrl+C` in that window (or File > Shut Down in the browser).

3.10.2 *Working in a notebook*

Step 1. New > Notebook, choose the **Python 3 (ipykernel)** kernel. Rename it from *Untitled* by clicking the title.

Step 2. Type `2 + 3` in the first cell and press `Shift+Enter`: the cell runs, `Out[1]:  5` appears and the cursor moves to a new cell.

Step 3. Paste the body of `hello.py` into a cell and run it; `input()` shows a text box under the cell.

Step 4. Change a cell's type to **Markdown** (toolbar drop-down, or `Esc` then `M`) and write `# Lab 1 -- Asha`; run it to see a heading.

Key	Action
`Shift+Enter`	run cell, move to next
`Ctrl+Enter`	run cell, stay
`Esc` / `Enter`	command mode (blue bar) / edit mode (green bar)
`A` / `B` (command mode)	insert cell above / below
`D` `D` (command mode)	delete cell
`M` / `Y` (command mode)	make Markdown / Code cell
Kernel > Restart & Run All	clear memory and run every cell top to bottom

Key idea: cells share memory, in the order you ran them

All cells talk to one running Python process, the **kernel**. The number in `In [7]` is the order of execution, not the position on the page. If you run cells out of order, the notebook can show results that a fresh run would not reproduce. Before submitting, always do Kernel > Restart & Run All.

JupyterLab (`pip install jupyterlab`, then `jupyter lab`) is the newer interface with file browser, tabs and terminal in one page; notebooks are identical in both.

3.11 Way 5: PyCharm

PyCharm by JetBrains is a full **IDE** (Integrated Development Environment) built only for Python: smart completion, error highlighting as you type, refactoring, a visual debugger, Git and test integration.

3.11.1 Install

1. Download from `https://www.jetbrains.com/pycharm/download/`. Since 2025 there is a single **PyCharm** download: its core features are free forever, and *Pro* features (Jupyter, web frameworks, databases) need a subscription. Students get Pro free with a college e-mail address via `https://www.jetbrains.com/community/education/`.
2. **Windows:** run the `.exe`; on the *Installation Options* page tick *Add "bin" folder to the PATH* and *.py* association if you like. **macOS:** open the `.dmg` (choose *Apple Silicon* or *Intel*) and drag PyCharm into *Applications*.
3. First launch: accept the agreement, choose a theme, skip plugins.

3.11.2 First project and run

Step 1. **New Project**. Set *Location* to `...\python-lab\lab1`. Under *Interpreter type* keep **Project venv** with *Python version* pointing to the Python 3.14 you installed (PyCharm finds it automatically; it can also download one for you). Untick "Create a welcome script" if you prefer. Click **Create**.

Step 2. In the **Project** pane on the left, right-click `lab1` → New > Python File → name it `hello` (PyCharm adds `.py`).

Step 3. Type the program. Red wavy underlines are errors; yellow are warnings. `Alt+Enter` offers a fix.

Step 4. Run: right-click in the editor → Run 'hello', or `Ctrl+Shift+F10` (Mac `Ctrl+Shift+R`), or the green triangle by the first line. Output and `input()` appear in the **Run** tool window at the bottom. After the first run, `Shift+F10` (Mac `Ctrl+R`) or the green triangle in the top toolbar re-runs it.

Step 5. **Debug:** click in the gutter beside line `total += i * i` to set a red breakpoint, then click the bug icon (`Shift+F9`). Step with `F8` and watch `i` and `total` change in the *Variables* pane.

The **Python Console** (bottom-left icon) gives a >>> prompt inside the project, and the **Terminal** tab opens a terminal with the project's venv already active, so `pip install numpy` lands in the right place. Packages can also be added through Settings > Project > Python Interpreter > +.

> **Note**
> If PyCharm says *No Python interpreter configured for the project*, click the interpreter name at the bottom-right of the status bar → Add New Interpreter > Add Local Interpreter and point it at the Python you installed.

3.12 Way 6: Visual Studio Code

VS Code is Microsoft's free, light, general-purpose editor. It becomes a strong Python IDE once you add the **Python extension**. It is the most widely used editor today and handles C, Java, web and Python in one tool.

3.12.1 Install

1. Download from `https://code.visualstudio.com`. **Windows:** run the *User Installer*; on *Select Additional Tasks* tick **Add "Open with Code" action** to both the file and directory context menus, and **Add to PATH**. **macOS:** unzip and drag *Visual Studio Code* to *Applications*; then in VS Code press `Cmd+Shift+P` and run **Shell Command: Install 'code' command in PATH**.

2. Open the **Extensions** view (`Ctrl+Shift+X`), search *Python*, and install the one published by **Microsoft**. It also brings *Pylance* (completion, error checking) and the *Python Debugger*.

3.12.2 First run

Step 1. File > Open Folder and choose `python-lab` (or type `code .` in a terminal in that folder). Click *Yes, I trust the authors*. Always open a *folder*, not a lone file: settings and the interpreter choice are stored per folder.

Step 2. New file icon in the Explorer pane → `hello.py`. The `.py` extension switches on Python support.

Step 3. **Select the interpreter:** `Ctrl+Shift+P` → **Python: Select Interpreter** → pick *Python 3.14.x* (or + *Create Virtual Environment... > Venv* to make a `.venv` for this folder, which is good practice). The chosen version shows at the bottom-right of the window.

Step 4. Run: click the **Run Python File** triangle at the top-right of the editor. VS Code opens the integrated **Terminal** (`Ctrl+``) and runs `python hello.py` there; type your name in that terminal.

Step 5. Other ways: select some lines and press `Shift+Enter` to run just those lines in a Python REPL; press `F5` → *Python File* to run under the **debugger** (set breakpoints by clicking left of the line numbers, step with `F10`/`F11`).

> **Watch out**
> The most common VS Code problem is the **wrong interpreter**: the file runs, but `import numpy` fails, or the version is not the one you expect. Look at the interpreter shown at the bottom-right and re-select it. Our `sys.executable` line tells you which Python actually ran.

3.13 Way 7: VS Code with Jupyter Notebooks

VS Code can open and run `.ipynb` notebooks itself, so you get Jupyter's cells inside a real editor with completion, a variable viewer and Git, and no browser or server window to keep open.

Step 1. Extensions view → install **Jupyter** (by Microsoft). It needs the Python extension from Way 6.

Step 2. Create a notebook: `Ctrl+Shift+P` → **Create: New Jupyter Notebook**, then save it as `lab1.ipynb` in your folder (or simply make a new file with the `.ipynb` extension). Existing notebooks open with a double-click.

Step 3. Click **Select Kernel** at the top-right → *Python Environments* → choose your Python 3.14 (or the `.venv`). If VS Code says the `ipykernel` package is required, click **Install**; it runs `pip install ipykernel` into that Python for you.

Step 4. Type code in a cell and press `Shift+Enter` (or the small triangle left of the cell). **+ Code** and **+ Markdown** add cells. `input()` opens a box at the top of the window.

Step 5. Click **Variables** in the notebook toolbar to see every variable, its type and value; **Restart** and **Run All** work as in the browser.

Bonus: cells inside an ordinary `.py` file

In any Python file, a line `# %%` starts a cell. VS Code shows *Run Cell* above each one and runs it in an **Interactive Window** beside the code. You get notebook-style exploration while keeping a plain `.py` file that runs normally from the terminal.

```
# %%
import math
r = 2.5
# %%
area = math.pi * r ** 2
print(f'area = {area:.2f}')
```

Note

Opening a notebook in VS Code does not need `pip install notebook`; it needs only `ipykernel` in the chosen Python. PyCharm offers the same notebook support in its Pro edition.

3.14 Choosing a Tool

Tool	Setup	Weight	Best for
Command Prompt / Terminal	none	tiny	quick checks, running finished scripts, `pip`
IDLE	none	tiny	first weeks, small programs, exams on lab PCs
Notepad + CMD	none	tiny	seeing that code is just text; no-install machines
Jupyter Notebook	`pip`	medium	data work, plots, lab records with notes
PyCharm	download	heavy	large projects, debugging, refactoring
VS Code	download + extension	medium	everyday coding in many languages
VS Code + Jupyter	+ extension	medium	notebooks with a real editor and Git

Recommendation for this course

Weeks 1–3: IDLE and the terminal, so you learn what the tools hide. From then on: VS Code for `.py` programs and VS Code (or Jupyter) notebooks for Unit IV data work. Learn one IDE well rather than four badly.

Draft

3.15 Troubleshooting

Symptom	Cause and fix
`'python' is not recognized as an internal or external command`	Not on PATH. Re-run installer → Modify → *Add Python to environment variables*; open a *new* Command Prompt. Try `py` meanwhile.
Typing `python` opens the Microsoft Store	Windows' placeholder alias. Install Python from python.org, or turn off Settings > Apps > Advanced app settings > App execution aliases > python.exe / python3.exe.
macOS: `zsh: command not found: python`	Use `python3`.
`pip` not recognized	Use `python -m pip ...` (`python3 -m pip` on macOS).
`ModuleNotFoundError` after installing	Installed into another Python. Print `sys.executable`; install with that exact path's `-m pip`; re-select the interpreter / kernel in your editor.
macOS: `SSL: CERTIFICATE_VERIFY_FAILED`	Run Applications > Python 3.14 > Install Certificates.command.
macOS: `error: externally-managed-environment`	You are using Homebrew's Python; create a venv (`python3 -m venv .venv`) and install there.
File saved as `hello.py.txt`	Notepad: *Save as type: All files*. Turn on file extensions in Explorer and rename.
`SyntaxError: invalid character "'"`	Smart (curly) quotes from TextEdit/Word. Use a plain-text editor; retype the quotes.
`IndentationError` / `TabError`	Mixed tabs and spaces. Use 4 spaces; in VS Code click *Spaces: 4* in the status bar → *Convert Indentation to Spaces*.
Window flashes and closes on double-click	Program finished. Run it from the terminal, or end with `input()`.
Jupyter: kernel not found / dead	`python -m pip install ipykernel`, then select the kernel again; Kernel > Restart.
VS Code: "Python is not installed"	Install Python first (Section 3.2 / 3.3), restart VS Code, then *Python: Select Interpreter*.
Program uses my `random.py` / `math.py`	Your file hides the standard module. Rename it and delete the `__pycache__` folder.

3.16 Summary

- Install Python 3 (64-bit) from python.org. Windows: tick **Add python.exe to PATH**. macOS: run **Install Certificates.command** and use `python3`.
- Verify with `python --version` (`py`, `python3`) in a *new* terminal; install packages with `python -m pip install ...`; isolate projects with `python -m venv .venv`.
- Seven front ends, one interpreter: terminal (`python file.py`), IDLE (`F5`), Notepad/TextEdit + terminal, Jupyter (`Shift+Enter` in the browser), PyCharm (`Shift+F10`), VS Code (Run triangle, `F5` to debug), VS Code notebooks (Select Kernel, `Shift+Enter`).
- When something "isn't installed", ask *which Python* is running: `sys.executable`, the editor's interpreter or kernel picker.

Dr. Hafiq